

Introduction to Iterators

INPUT/OUTPUT AND STREAMS IN JAVA



Alex Liu

Software Development Engineer

Working with collections

- Example `HashSet` for the rest of the video

```
import java.util.HashSet;

public class SampleHashSetData {
    public static void main(String[] args) {
        HashSet<String> fruits = new HashSet<>();
        fruits.add("Apple");
        fruits.add("Banana");
    }
}
```

```
{Apple, Banana}
```

Use For-Each loop

- **For-Each loop**
 - Format: `for (type x: collection){}`
 - Automatically processes each element
 - Works for Multiple Collections: `List` , `Set` , `Map` and more
- Example: use sample set `fruits` : `[Apple, Banana]` .

```
for (String x : fruits) {  
    System.out.println(x);  
}
```

```
Apple  
Banana
```

Traversing a collection with Iterator

- Use `Iterator<>` : Supports `List` , `Set` , `Queue` and more
- Import `Iterator` class

```
import java.util.Iterator;

// Create `Iterator` object named `it` for sample set `fruits`
Iterator<String> it = fruits.iterator();
// Use `.hasNext()` method checks if more elements exist
while (it.hasNext()) {
    // User `.next()` retrieves the element
    System.out.print(it.next());}
```

Apple Banana

Removing elements with Iterator

- Use `.remove()` to remove element with `Iterator`

```
Iterator<String> it = fruits.iterator();
while (it.hasNext()) {
    String fruit = it.next();
    if (fruit.startsWith("A")) {
        // Remove element if element startsWith `A`
        it.remove();
    }
}
// fruits now contains only "Banana"
```

Removing elements with Iterator(Continued)

- Before Removal:

```
System.out.println(fruits);
```

```
[Apple, Banana]
```

- After Removal:

```
System.out.println(fruits);
```

```
[Banana]
```

- Ensures safe deletion and prevent
`ConcurrentModificationException`



Summary

- For-Each loop

- Support

- List (ArrayList , LinkedList)
 - Set (HashSet , TreeSet ,
LinkedHashSet)
 - Queue (PriorityQueue)
 - Stack (Stack)

- Not Support

- Map
 - Element Removal

- Iterator

- Support

- List (ArrayList , LinkedList)
 - Set (HashSet , TreeSet ,
LinkedHashSet)
 - Queue (PriorityQueue)
 - Stack (Stack)
 - Map (Must use `keySet().iterator` ,
`values().iterator` or
`entrySet().iterator`)
 - Element Removal

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Modifying lists with ListIterator

INPUT/OUTPUT AND STREAMS IN JAVA



Alex Liu

Software Development Engineer

Iterator vs ListIterator

Feature	Iterator	ListIterator
Works on all collections	✓ Yes	✗ No (Lists only)
Forward traversal	✓ Yes	✓ Yes
Backward traversal	✗ No	✓ Yes
Modify elements (<code>.set()</code>)	✗ No	✓ Yes
Insert elements (<code>.add()</code>)	✗ No	✓ Yes
Access next/previous index	✗ No	✓ Yes

Sample ArrayList

```
import java.util.ArrayList;
public class SampleData {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");
    }
}
```

[Alice, Bob, Charlie]

Traversing a list with ListIterator

```
// Import ListIterator class
import java.util.ListIterator;

// Create ListIterator object for sample list `names`
ListIterator<String> it = names.listIterator();
// Use .hasNext() and `.next()` to iterate the list and print line by line
while (it.hasNext()) {
    System.out.println(it.next());
}
```

```
Alice
Bob
Charlie
```

Moving backward with ListIterator

- Use `.previous()` to move backward in the list

```
// Create ListIterator object for sample list `names`
ListIterator<String> it = names.listIterator(names.size());
// Use .hasPrevious() to check if the iterator reach the beginning of the list
while (it.hasPrevious()) {
    // Use .previous() to retrieve the element in reverse order
    System.out.println(it.previous());}
```

- Print element in reserve order, output:

```
Charlie
Bob
Alice
```

Modifying elements during iteration

- Using the `.set()` method

```
// Create ListIterator object for sample list names
ListIterator<String> it = names.listIterator();
// Iterate the list and modify element using .set()
while (it.hasNext()) {
    String name = it.next();
    if ("Bob".equals(name)) it.set("Bobby");
}
```

- Updated `names` list:

```
[Alice, Bobby, Charlie]
```

Adding elements during iteration

- Use `.add()` to add elements

```
ListIterator<String> it = names.listIterator();

// Iterate the list and insert element using .add()
while (it.hasNext()) {
    String name = it.next();
    if ("Charlie".equals(name)) it.add("David");
}
```

- Updated `names` list:

```
[Alice, Bobby, Charlie, David]
```

Summary

- `ListIterator` extends `Iterator`
 - Supports both forward and backward traversal
 - Allows modifying elements while iterating
- Key Methods Recap
 - `.next()` / `.previous()` : Navigate forward and backward
 - `.set()` : Modify the current element
 - `.add()` : Insert elements dynamically
 - `.remove()` : Safely delete elements
 - Works only on Lists (`ArrayList` , `LinkedList` , etc.)

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Introduction to Streams

INPUT/OUTPUT AND STREAMS IN JAVA



Alex Liu

Software Development Engineer

Working with Streams

- Java Streams
 - Process data collections **functionally** and Improve **performance** for **large data**
 - Supported Collections:
 - Lists (e.g. `ArrayList` , `LinkedList`)
 - Sets (e.g. `HashSet` , `TreeSet`)
 - Maps (`Keys` / `Values` as `Streams`) (e.g., `HashMap` , `TreeMap`)
 - Arrays (via `Arrays.stream()`)
- Example `ArrayList` named `names` to use for the rest of the video

```
names: ["Alice", "Bob", "Charlie", "David"]
```

Using lambda expressions with Stream

- Lambda Expressions
 - Syntax: `(parameters) -> { expression }`

```
names.forEach(  
    // `name` is a parameter representing each element in the list  
    name ->  
        // This is the expression executed for each element.  
        System.out.println(name));
```

```
Alice  
Bob  
Charlie  
David
```

Creating a Stream

- Convert data collection to `Stream`
 - Use `.stream()` on a collection
 - Supports `List`, `Set` and `Queue`
- Import `Stream` class

```
import java.util.stream.Stream;
```

- Converted sample list `names` to streams and use `.foreach()` to access each element.

```
Stream<String> stream = names.stream();  
stream.forEach(name -> System.out.println(name));
```

Filtering data with Streams

- Use `.filter()` to select elements
 - Keeps only **matching elements**
- Converted sample list `names` to streams

```
Stream<String> stream = names.stream();
```

- Use `.filter()` to match elements starts with `A` and use `.forEach()` to print them

```
stream
    .filter(name -> name.startsWith("A"))
    .forEach(name -> System.out.println(name));
```

- Filter result: `Alice`

Counting elements with Streams

- Use `.count()` to count elements
 - Returns value in `Long` format
- Converted sample list `names` to streams

```
Stream<String> stream = names.stream();
```

- Use `.filter()` to match elements starts with `B` and use `.count()` to count them

```
long count = names.stream()  
    .filter(name -> name.startsWith("B"))  
    .count();
```

- Count result: 1 (only 1 element `Bob` start with `B`)

When to use Streams

- When to Use Streams :
 - For large datasets: Reduces performance overhead
 - For functional operations: More readable than loops
 - For filtering and transformations: Simplifies code
- When **NOT** to Use Streams :
 - For modifying original collections
 - For simple loops with side effects

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA

Data transformation with Streams

INPUT/OUTPUT AND STREAMS IN JAVA



Alex Liu

Software Development Engineer

Data transformation with streams

- Data transformation with streams
 - Convert data formats, aggregate and filter information
 - Store results in new data collection type(eg. from `List` to `Set`)
 - Supported Collections:
 - Lists (e.g. `ArrayList` , `LinkedList`)
 - Sets (e.g. `HashSet` , `TreeSet`)
 - Maps (`Keys` / `Values` as `Streams`) (e.g., `HashMap` , `TreeMap`)
 - Arrays (via `Arrays.stream()`)
- Example `ArrayList` named `names` to use for the rest of the video

```
names: ["Alice", "Bob", "Charlie", "David"]
```

Using Stream to transform elements

- Preparation: import required class `Set`, `Collectors` and `Stream`

```
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.Stream;

// Convert the `ArrayList` we created earlier `names` to a `Stream`
Stream<String> namesStream = names.stream();
```

- Current data: `[Alice, Bob, Charlie, David]`

Using Stream to transform elements (Continued)

```
// Use `.map()` to convert each element to upper case  
Stream<String> upperCaseNameStream = namesStream.map(  
    name -> name.toUpperCase())
```

- Current data: [ALICE, BOB, CHARLIE, DAVID]

```
// use `.collect()` and `Collectors.toSet()` to store the transformed stream data  
upperCaseNameStream.collect(Collectors.toSet())
```

- Final data:

```
{CHARLIE, ALICE, BOB, DAVID}
```

Using Stream for aggregation

- The `.reduce()` method
 - **Aggregates elements** into a single result
 - Useful for **summation, concatenation, or finding min/max**
- Use `.reduce()` takes two inputs
 - **Initial Value:** the starting value for the accumulation, e.g., `0`
 - **Binary Operator:** A function that takes two arguments
 - `(x, y) -> x + y` means:
 - `x` represents the accumulated total
 - `y` represents the current element being processed
 - The operation adds `y` to `x`, continuing until all elements are processed

Using Stream for aggregation: Example

- Convert the `ArrayList` `names` to a `Stream` object.

```
Stream<String> stream = names.stream();
```

- Use `.map()` to convert name to its length and use `.reduce()` to add them up together

```
stream
    .map(name -> name.length())
    .reduce(0, (sum, length) -> sum + length);
```

- Result: `20`

Let's practice!

INPUT/OUTPUT AND STREAMS IN JAVA